Model Deployment: REST API with Flask and Containerization with Docker

1. app.py (Full code)

This single Python file trains the model, saves it, and deploys it through Flask.

app.py

```
from flask import Flask, request, jsonify from sklearn.datasets import load_iris from sklearn.ensemble import RandomForestClassifier import pickle import os app = Flask(__name__) if not os.path.exists("model.pkl"): iris = load_iris() model = RandomForestClassifier(random_state=42) model.fit(iris.data, iris.target) with open("model.pkl", "wb") as f: pickle.dump(model, f) with open("model.pkl", "rb") as f: model = pickle.load(f) @app.route("/", methods=["GET"]) def home(): return jsonify({ "message": "Machine Learning API is running" }) @app.route("/predict", methods=["POST"]) def predict(): try: data = request.get_json() features = data["features"] if len(features) != 4: return jsonify({ "error": "Provide exactly 4 features" }), 400
```

2. requirements.txt

```
flask scikit-learn numpy
```

3. Dockerfile

```
FROM python:3.11-slim WORKDIR /app COPY requirements.txt . RUN pip install --no-cache-dir -r requirements.txt COPY app.py . EXPOSE 5000 CMD ["python", "app.py"]
```

4. Run the application

Install dependencies:

```
pip install flask scikit-learn numpy requests
```

Start the Flask API:

```
python app.py
```

Open **http://127.0.0.1:5000/** in your browser to check the API.

5. Test the prediction API

Run this in a separate Python file or terminal session:

test_api.py

```
import requests url = "http://127.0.0.1:5000/predict" data = { "features": [5.1, 3.5, 1.4, 0.2] } response = requests.post(url, json=data) print(response.json())
```

Expected output:

```
{ "prediction": "setosa", "class_id": 0 }
```

6. Deploy using Docker

Run these commands from the folder containing `app.py`, `requirements.txt`, and `Dockerfile`.

```
docker build -t iris-ml-api . docker run -p 5000:5000 iris-ml-api
```

Test the running container using the same prediction request above.

> **Result:**
> The Iris classification model was trained and saved, deployed through a Flask REST API, and packaged into a Docker container. The API accepts four flower measurements and returns the predicted species as JSON.